

BIG-O QUICK REFERENCE

Class	Name	Typical example
O(1)	constant	dict lookup, array index
O(log n)	logarithmic	binary search, heap push/pop
O(n)	linear	one pass, hash-map solutions
O(n log n)	linearithmic	sorting, merge sort, heap of n
O(n²)	quadratic	nested loops, bubble sort
O(2^n), O(n!)	exponential, factorial	subsets, permutations

- n = 10^5 needs O(n log n) or better; n = 10^3 allows O(n²); n <= 20 allows O(2^n).
- Drop constants and lower-order terms; state **time and space** separately; recursion uses stack space O(depth).
- Amortised O(1): dynamic array append, hash-table insert.

DATA STRUCTURES: WHAT EACH IS GOOD AT

Structure	Access	Search	Insert/delete	Use for
array / list	O(1)	O(n)	O(n) middle, O(1) end	indexed data
hash map / set		O(1) avg	O(1) avg	counting, lookup, dedupe
linked list	O(n)	O(n)	O(1) at node	fast splice, queues
stack (list)	top O(1)		push/pop O(1)	undo, parsing, DFS
queue (deque)			O(1) both ends	BFS, windows
heap (heapq)	min O(1)		O(log n)	top-k, scheduling
BST balanced		O(log n)	O(log n)	ordered data
trie		O(len)	O(len)	prefix search
graph (adj. list)				networks, dependencies

- Sorting: Python sorted is Timsort, O(n log n), stable. Merge sort O(n log n) stable; quicksort O(n log n) average, O(n²) worst; counting sort O(n + k).

CLUE -> PATTERN

If the problem says	Think
pair / complement / seen before / count	hash map or set
sorted array, pair or triplet sum	two pointers
longest / shortest substring or subarray with a rule	sliding window
sum of a range, many queries	prefix sums
sorted, or "minimum value such that"	binary search (on answer)
matching brackets, next greater element	stack, monotonic stack
shortest path, levels, fewest steps	BFS
all paths, components, tree recursion	DFS
top k, k-th largest, merge k sorted	heap
overlapping intervals	sort by start, then merge
count ways, min cost, choices at each step	dynamic programming
all subsets, permutations, combinations	backtracking
connected groups, cycle in undirected graph	union-find

INTERVIEW ROUTINE

1. Restate; ask about input size, duplicates, negatives, empty input, sorted?
 2. Say a brute force and its cost; then improve (hash map? sort? pointers?).
 3. State the plan; code; **trace** on a small example.
 4. Edge cases: empty, one element, all same, duplicates, negatives, large n.
 5. State time and space. Target: easy in ~10 minutes, medium in ~20.
- Learn ~8 patterns well; re-solve problems from a blank page a week later.

PYTHON HELPERS FOR DSA

```
from collections import Counter, defaultdict, deque
import heapq, bisect, math
float("inf"); math.inf; sys.setrecursionlimit(10**6)
sorted(xs, key=lambda x: (-x[1], x[0]))
bisect.bisect_left(xs, x) # lower bound
divmod(a, b); a // b; a % b # floor division
```

heapq is a min-heap: push -x for a max-heap, tuples (priority, item) for ordering. deque.popleft() is O(1); list.pop(0) is O(n).

HASH MAP: TWO-SUM, COUNTING

```
def two_sum(nums, t):
    seen = {}
    for i, x in enumerate(nums):
        if t - x in seen:
            return [seen[t - x], i]
    seen[x] = i
# subarray sum == k (prefix sums + map)
cnt = Counter({0: 1}); s = res = 0
for x in nums:
    s += x
    res += cnt[s - k]
    cnt[s] += 1
```

TWO POINTERS (SORTED ARRAY)

```
l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == t: return l, r
    if s < t: l += 1
    else: r -= 1
```

Also: remove duplicates in place (slow/fast pointer), reverse, merge two sorted lists, container with most water, 3-sum (sort, fix one, two pointers).

SLIDING WINDOW

```
# longest substring without repeats
last = {}; l = best = 0
for r, ch in enumerate(s):
    if ch in last and last[ch] >= l:
        l = last[ch] + 1
    last[ch] = r
    best = max(best, r - l + 1)
```

Fixed size k: add a[r], drop a[r - k]. Variable: expand right, shrink left while the rule is broken.

BINARY SEARCH

```
def bs(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo
```

On the answer: binary search `lo..hi` with `ok(mid)` monotonic (min capacity, min speed). Avoid `(lo + hi)` overflow in other languages.

STACK AND MONOTONIC STACK

```
def valid(s):
    pair = {"(": ")", "[": "]", "{": "}"
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]: return False
            else: st.append(c)
    return not st
# next greater element
res = [-1] * n; st = []
for i, x in enumerate(a):
    while st and a[st[-1]] < x: res[st.pop()] = x
    st.append(i)
```

BFS AND DFS (GRID / GRAPH)

```
def bfs(start, adj):
    seen = {start}; q = deque([(start, 0)])
    while q:
        node, d = q.popleft()
        for nb in adj[node]:
            if nb not in seen:
                seen.add(nb); q.append((nb, d + 1))
def dfs(node, adj, seen):
    seen.add(node)
    for nb in adj[node]:
        if nb not in seen: dfs(nb, adj, seen)
```

Grid neighbours: for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)), check bounds. BFS = shortest path in an unweighted graph; mark visited when enqueued. Cycle in a directed graph: DFS with colours or topological sort (Kahn: in-degrees).

HEAP: TOP-K, MERGE INTERVALS

```
heapq.nlargest(k, xs)
h = []
for x in xs:
    heapq.heappush(h, x)
    if len(h) > k: heapq.heappop(h)
# merge intervals
out = []
for s, e in sorted(iv):
    if out and s <= out[-1][1]:
        out[-1][1] = max(out[-1][1], e)
    else: out.append([s, e])
```

DYNAMIC PROGRAMMING

```
from functools import lru_cache
@lru_cache(None)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
# coin change: fewest coins
dp = [0] + [inf] * amt
for a in range(1, amt + 1):
    for c in coins:
        if c <= a: dp[a] = min(dp[a], dp[a - c] + 1)
```

Recipe: define the state, the recurrence, the base case, the order. Top-down (memoised recursion) or bottom-up table. Classics: climbing stairs, knapsack, longest increasing subsequence ($O(n \log n)$) with bisect), edit distance, longest common subsequence.

BACKTRACKING, LINKED LIST, TREE

```
def subsets(a):
    res = []
    def go(i, cur):
        if i == len(a): res.append(cur[:]); return
        go(i + 1, cur)
        cur.append(a[i]); go(i + 1, cur); cur.pop()
    go(0, []); return res
# reverse linked list
prev = None
while head:
    head.next, prev, head = prev, head, head.next
# tree height
def h(t): return 0 if not t else 1 + max(h(t.left), h(t.right))
```

Inorder of a BST is sorted. Level order = BFS with a queue.

GRAPH EXTRAS: DIJKSTRA, TOPOLOGICAL SORT, UNION-FIND

```
# Dijkstra (non-negative weights),  $O((V+E) \log V)$ 
dist = {src: 0}; h = [(0, src)]
while h:
    d, u = heapq.heappop(h)
    if d > dist.get(u, inf): continue
    for v, w in adj[u]:
        if d + w < dist.get(v, inf):
            dist[v] = d + w; heapq.heappush(h, (d + w, v))
# union-find
def find(x):
    while p[x] != x: p[x] = p[p[x]]; x = p[x]
    return x
def union(a, b): p[find(a)] = find(b)
```

Topological order (Kahn): queue nodes with in-degree 0, pop, decrement neighbours; if fewer than V nodes output, there is a cycle. Weighted graph with negative edges: Bellman-Ford.

BITS, MATH, STRINGS

- $x \& 1$ odd; $x >> 1$ half; $x \& (x - 1)$ clears lowest set bit; $x \wedge x == 0$ (single number); $1 \ll k$.
- Primes: sieve of Eratosthenes $O(n \log \log n)$. `math.gcd`, `pow(a, b, mod)`.
- Palindrome expand-around-centre $O(n^2)$; anagram by sorted key or Counter; strings are immutable, build with a list and `join`.
- Matrix: transpose `zip(*m)`, rotate 90 clockwise `[list(r)[::-1] for r in zip(*m)]`; prefix sums per row for 2-D range sums.

COMMON MISTAKES

TRAP Off-by-one in loops and binary search; infinite loop when `lo/hi` do not move. Mutating a list while iterating; appending `cur` instead of `cur[:]` in backtracking. Forgetting to mark visited before enqueueing (BFS) or the base case (recursion). Using `list.pop(0)` as a queue, `+=` on strings in a loop, or a default mutable argument.

- Test: empty input, one element, duplicates, negative numbers, a sorted and a reverse-sorted input.

SAY IT IN 20 SECONDS

- "Brute force is $O(n^2)$; a hash map makes it $O(n)$ time, $O(n)$ space."
- "Sliding window: the window is valid or I shrink it."
- "BFS finds the shortest path in an unweighted graph."
- "Dynamic programming = state, recurrence, base case."